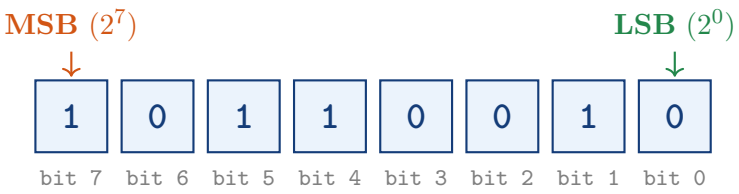
The Ultimate Guide to Bit Manipulation in Competitive Programming

From Binary Foundations to Grandmaster Techniques



A complete, self-contained curriculum covering Two's Complement arithmetic, Bitwise Intrinsics, SWAR, Bit-by-Bit Contribution, Submask Enumeration, Gosper's Hack, Gray Codes, Bitmask DP, Sum Over Subsets (SOS) DP, Bit-Parallel `std::bitset`, Binary Tries, and Linear XOR Basis over $\mathbb{F}_2$.

Compiled with `pdflatex` • Designed for Mastery

Contents

1 Prerequisites & Binary Foundations **5**
- 1.1 Place Value & The Base-2 System . . . 5
  - 1.1.1 Essential Powers of 2 to Memorize for CP . . . 6
- 1.2 The Fundamental Geometric Sum Identity . . . 6
- 1.3 Fixed-Width Integers & Two's Complement . . . 6
  - 1.3.1 Why $-x = \sim x + 1$ Always Holds . . . 7

2 The 6 Fundamental Bitwise Operators **9**
- 2.1 Truth Tables & Mental Models . . . 9
- 2.2 Critical C++ Traps & Undefined Behavior (UB) . . . 10

3 Elementary Bitmasks & Single-Bit Hacks **11**
- 3.1 The Big Four Bit Operations . . . 11
- 3.2 Brian Kernighan's Trick: `x & (x - 1)` . . . 11
  - 3.2.1 Direct Applications of `x & (x - 1)` . . . 12
- 3.3 Isolating the Lowest Set Bit: `x & -x` (The Fenwick Tree Trick) . . . 13
- 3.4 Complete Cheat Sheet of Single-Word Bit Hacks . . . 14

4 GCC Built-in Intrinsics, C++20 `<bit>`, & SWAR **15**
- 4.1 The Essential GCC Intrinsics . . . 15
  - 4.1.1 Computing $\lfloor \log_2(x) \rfloor$ and Highest Power of 2 in $O(1)$ . . . 15
  - 4.1.2 Modern C++20 `<bit>` Equivalents . . . 15
- 4.2 Under the Hood: SWAR (SIMD Within A Register) . . . 16

5 The Algebra of XOR & Bitwise Contribution Technique **19**
- 5.1 The Five Algebraic Laws of XOR . . . 19
  - 5.1.1 Classic Application: Finding Two Unique Numbers . . . 19
- 5.2 Fundamental Identities Connecting Addition and Bitwise Ops . . . 20
- 5.3 The Bit-by-Bit Contribution Technique . . . 20

6 Bitmasks for Sets, Submask Iteration, Gosper's Hack, & Gray Codes **23**
- 6.1 Set Theory via Bitmasks in $O(1)$ . . . 23
- 6.2 Iterating Over All Submasks of a Mask in $O(3^n)$ . . . 23
  - 6.2.1 Why Iterating Submasks for All Masks is $O(3^n)$, NOT $O(4^n)$! . . . 24
- 6.3 Gosper's Hack: Generating All k-Bit Masks in $O(\binom{n}{k})$ . . . 25
- 6.4 Gray Codes (Reflected Binary Code) . . . 26

7 Bitmask Dynamic Programming & SOS DP **27**
- 7.1 Standard Bitmask DP Patterns . . . 27
  - 7.1.1 Pattern 1: Traveling Salesperson Problem (TSP) in $O(N^2 2^N)$ . . . 27
  - 7.1.2 Pattern 2: Bipartite Assignment / Matching in $O(N2^N)$ . . . 28
- 7.2 Sum Over Subsets (SOS) DP: From $O(3^N)$ to $O(N2^N)$ . . . 28

7.2.1 The 7-Line In-Place 1D SOS DP Implementation 29
7.2.2 OR / AND Convolutions via SOS DP (Fast Zeta & Möbius Transform) . 30

8 Advanced Data Structures: `std::bitset`, Binary Tries, & XOR Basis 31
8.1 Bit-Parallel Speedups with `std::bitset` ($O(N/64)$) 31
8.1.1 Undocumented GCC Power Functions on `std::bitset` 31
8.1.2 Classic Bitset Applications 31
8.2 The Binary Trie (0-1 Trie) 32
8.2.1 Complete Production C++ Binary Trie Template 32
8.3 Linear XOR Basis (Gaussian Elimination over $\mathbb{F}_2^d$) 33
8.3.1 Upper-Triangular Echelon Form & Online Insertion 34
8.3.2 The Five Fundamental Theorems of XOR Basis 34
8.3.3 Complete Grandmaster C++ `XorBasis` Struct 35

9 Problem-Solving Decision Matrix & Practice Roadmap 37
9.1 Curated Practice Problem Set (Sorted by Rating) 37

Chapter 1

Prerequisites & Binary Foundations

Before touching a single bitwise trick, we need rock-solid intuition for how integers live inside silicon. Every advanced technique in this book—from Fenwick trees to XOR Basis—is a direct consequence of how numbers are represented in base-2.

1.1 Place Value & The Base-2 System

In everyday life, we use **base-10 (decimal)**. When we write the number 305_{10}, we really mean:

$$305_{10} = 3 \times 10^2 + 0 \times 10^1 + 5 \times 10^0$$

Computers operate using transistors that have two stable voltage states: **OFF (0)** and **ON (1)**. Consequently, all data is represented in **base-2 (binary)**, where each position i (starting from 0 on the right) represents a weight of 2^i.

Given a W-bit unsigned binary number with digits $b_{W-1}b_{W-2}\ldots b_1b_0$ where each $b_i \in \{0, 1\}$, its numerical value is:

$$V = \sum_{i=0}^{W-1} b_i \cdot 2^i$$

Intuition & Mental Model: 0-Based Indexing from Right to Left

Always index bits starting at **bit 0 on the far right**, moving left! Why? Because the i-th bit has weight 2^i.

- **LSB (Least Significant Bit):** Bit 0 (weight $2^0 = 1$). It determines whether the number is odd or even.
- **MSB (Most Significant Bit):** The highest bit position (or highest set bit). It determines the overall magnitude (or sign) of the number.

Step-by-Step Dry Run: Converting Binary 110101_2 to Decimal

Let us index the bits of 110101_2 from right to left:

Bit Index i :	5	4	3	2	1	0
Weight 2^i :	32	16	8	4	2	1
Bit b_i :	1	1	0	1	0	1

Summing the weights where $b_i = 1$:

$$110101_2 = 32 + 16 + 4 + 1 = 53_{10}.$$

1.1.1 Essential Powers of 2 to Memorize for CP

In Competitive Programming, time and memory limits are closely tied to powers of 2. Knowing these benchmarks allows you to immediately spot whether a bitmask algorithm will pass within the 1-second limit ($\approx 10^8$ operations):

Power	Exact Value	Approximation	CP Significance
2^{10}	$1,024$	$\approx 10^3$	1 Kilobyte (KB)
2^{16}	$65,536$	$\approx 6.5 \times 10^4$	Max 2^N for $O(3^N)$ submask DP ($3^{15} \approx 1.4 \times 10^7$)
2^{20}	$1,048,576$	$\approx 10^6$	Max 2^N for $O(N2^N)$ Bitmask / SOS DP ($N \leq 20$)
2^{30}	$1,073,741,824$	$\approx 10^9$	Fits inside signed 32-bit `int`
$2^{31}-1$	$2,147,483,647$	$\approx 2.14 \times 10^9$	`INT_MAX` (Signed 32-bit maximum)
2^{62}	4.61×10^{18}	$\approx 4.6 \times 10^{18}$	Safe infinity for 64-bit `long long`
$2^{63}-1$	9.22×10^{18}	$\approx 9.22 \times 10^{18}$	`LLONG_MAX` (Signed 64-bit maximum)

1.2 The Fundamental Geometric Sum Identity

One algebraic identity governs almost every bit-borrowing and greedy bit manipulation proof:

Key Identity / Theorem: Geometric Series of Powers of 2

For any non-negative integer k:

$$\sum_{i=0}^{k-1} 2^i = 2^0 + 2^1 + 2^2 + \cdots + 2^{k-1} = 2^k - 1$$

In binary, 2^k is a 1 followed by k zeros, while $2^k - 1$ is simply k consecutive ones:

$$2^k = 1\underbrace{00\ldots0}_{k \text{ zeros}}{}_2 \quad \Longleftrightarrow \quad 2^k - 1 = 0\underbrace{11\ldots1}_{k \text{ ones}}{}_2$$

Why is this huge for greedy algorithms? It says that the k-th bit alone (2^k) is strictly greater than the sum of *all smaller bits combined* ($2^k - 1$). Therefore, if you want to maximize a quantity (like maximum XOR pair), securing a 1 at bit k is always strictly better than getting 1s at every single bit position below k!

1.3 Fixed-Width Integers & Two's Complement

In C++, a standard `int` occupies $W = 32$ bits, and a `long long` occupies $W = 64$ bits. How does a computer store negative numbers using only 0s and 1s?

Early engineers tried reserving the MSB as a simple sign flag (Sign-Magnitude), where $0001_2 = +1$ and $1001_2 = -1$. However, that created two problems:

1. There were two zeros ($+0 = 0000_2$ and $-0 = 1000_2$).
2. Standard binary addition circuits failed when adding positive and negative numbers: $0001_2 + 1001_2 = 1010_2 (-2)$, which is wrong!

Modern CPUs solve both problems simultaneously using **Two's Complement**.

Intuition & Mental Model: Two's Complement via a Negative MSB Weight

In a W-bit signed integer using Two's Complement, the lower $W-1$ bits have their normal positive weights $(+2^0, +2^1, \ldots, +2^{W-2})$, while the **Most Significant Bit (b_{W-1}) has a NEGATIVE weight of -2^{W-1}**:

$$V_{\text{signed}} = -b_{W-1} \cdot 2^{W-1} + \sum_{i=0}^{W-2} b_i \cdot 2^i$$

Let us see how a 4-bit signed integer ($W = 4$, where the MSB has weight $-2^3 = -8$) works:

Binary	-8	$+4$	$+2$	$+1$	**Decimal Calculation**
`0111`	0	1	1	1	$0+4+2+1=+7$ (Max positive)
`0001`	0	0	0	1	$0+0+0+1=+1$
`0000`	0	0	0	0	0
`1111`	1	1	1	1	$-8+4+2+1=-1$ (All ones is ALWAYS -1!)
`1110`	1	1	1	0	$-8+4+2+0=-2$
`1011`	1	0	1	1	$-8+0+2+1=-5$
`1000`	1	0	0	0	$-8+0+0+0=-8$ (Min negative)

1.3.1 Why $-x = \sim x + 1$ Always Holds

Take any W-bit integer x. What happens when we flip every bit ($\sim x$)? Since $x + (\sim x)$ has a 1 in every single bit position, it equals the all-ones binary number $111\ldots11_2$. And as we just saw in the table above, all-ones in Two's Complement is always equal to -1!

$$x + (\sim x) = -1 \quad \Longrightarrow \quad -x = \sim x + 1 \quad \text{and} \quad \sim x = -x - 1.$$

Memorize both forms:

- **To negate x in binary:** Invert all bits and add 1 ($-x = \sim x + 1$).
- **Bitwise NOT value:** $\sim x = -(x+1)$. For example, $\sim 0 = -1$ and $\sim 5 = -6$.

Chapter 2

The 6 Fundamental Bitwise Operators

Bitwise operators act on integers by applying a boolean operation to each bit position $i \in [0, W-1]$ independently and in parallel.

2.1 Truth Tables & Mental Models

a	b	$a \,\&\, b$ (**AND**)	$a \mid b$ (**OR**)	$a \oplus b$ (**XOR**)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Instead of memorizing static truth tables, think of each operator as an active tool:

1. **Bitwise AND (`&`) — The Filter / Mask:**
   - $x \,\&\, 0 = 0$ (blocks the bit), $x \,\&\, 1 = x$ (lets the bit pass through).
   - Think of AND as a stencil: wherever your mask has 1s, the original bits of x survive; wherever your mask has 0s, the bits are wiped to 0.
2. **Bitwise OR (`|`) — The Forced Setter:**
   - $x \mid 0 = x$ (leaves the bit alone), $x \mid 1 = 1$ (forces the bit ON).
3. **Bitwise XOR (`^`) — The Conditional Toggle / Parity Adder:**
   - $x \oplus 0 = x$ (leaves the bit alone), $x \oplus 1 = \sim x$ (flips the bit!).
   - Equivalently, $a \oplus b = (a + b) \bmod 2$: it is addition in base-2 *without carrying*.
4. **Bitwise NOT (`~`) — The Inverter:**
   - Flips every $0 \leftrightarrow 1$ across the entire register width W.
5. **Left Shift (`x << k`) — Multiply by 2^k:**
   - Shifts all bits of x to the left by k positions, filling the vacated right positions with 0s.
   - Mathematically: $x \ll k = x \cdot 2^k$ (modulo 2^W).
6. **Right Shift (`x >> k`) — Divide by 2^k (Floor):**

- Shifts all bits of x to the right by k positions, discarding the lowest k bits.
- Mathematically: $x \gg k = \lfloor x/2^k \rfloor$. (Note: for negative numbers, arithmetic right shift floors toward $-\infty$, so $-5 \gg 1 = -3$, whereas C++ integer division $-5/2 = -2$ truncates toward 0!)

2.2 Critical C++ Traps & Undefined Behavior (UB)

More competitive programmers lose points on bitwise problems due to C++ language traps than algorithmic bugs. Read these three traps carefully!

Compiler & Overflow Trap: 1. The 32-Bit Literal Shift Overflow (`1 « k` vs `1LL « k`)

In C++, the literal `1` has type `int` (32-bit signed). Even if you assign the result to a 64-bit `long long`:

```
long long mask = 1 << 40; // UNDEFINED BEHAVIOR! Overflows 32-bit int FIRST!
```

Shifting a 32-bit `int` by $k \geq 31$ is Undefined Behavior! To shift in 64-bit space, you **must** make the left operand a 64-bit literal using `1LL` or `1ULL`:

```
long long mask = 1LL << 40; // SAFE! Evaluates in 64-bit long long.
```

Compiler & Overflow Trap: 2. The Operator Precedence Disaster

In C++, the bitwise operators `&`, `^`, and `|` have **lower precedence** than relational comparison operators (`==`, `!=`, `<`, `>`)!

```
if (x & 1 == 0)   // BUG! Parsed by compiler as: if (x & (1 == 0)) -> if (x & 0) -> ALWAYS FALSE!
if ((x & 1) == 0) // CORRECT! Always wrap bitwise expressions in parentheses.
```

Similarly, `«` and `»` have lower precedence than `+` and `-`:

```
int val = 1 << k - 1;   // BUG! Parsed as: 1 << (k - 1)
int mask = (1 << k) - 1; // CORRECT!
```

Compiler & Overflow Trap: 3. Shifting by $\geq$ Bit-Width or Negative Amounts

If `x` is a 32-bit `int` and $k = 32$, evaluating `x « 32` or `x » 32` is strict Undefined Behavior in C++. On x86 CPUs, the hardware shift instruction masks the shift count by 31 (so `1 « 32` evaluates to `1 « 0 = 1` instead of 0!). Always guard shifts or use `1ULL « k` when $k \in [0, 63]$.

Chapter 3

Elementary Bitmasks & Single-Bit Hacks

Let us build our toolkit for inspecting and surgical manipulation of individual bits. Assume x is a 64-bit integer and $k \in [0, 62]$ is the 0-based bit index.

3.1 The Big Four Bit Operations

Operation	C++ Expression	How It Works
Check k-th bit	`(x >> k) & 1LL`	Shifts bit k to position 0 and masks all other bits.
Set k-th bit (to 1)	`x | (1LL << k)`	ORs with a mask that has a 1 only at bit k.
Clear k-th bit (to 0)	`x & ~(1LL << k)`	ANDs with a mask that has 1s everywhere *except* bit k.
Toggle k-th bit	`x ^ (1LL << k)`	XORs with a 1 at bit k, flipping $0 \leftrightarrow 1$.

Step-by-Step Dry Run: Clearing the 3rd Bit of $x = 29$ (`00011101`)

Suppose $x = 29 = 00011101_2$ and we want to clear bit $k = 3$ (which represents $2^3 = 8$, currently 1):

1. Construct single-bit mask: `1 << 3` $= 00001000_2$.
2. Invert the mask: `~(1 << 3)` $= 11110111_2$. Notice every bit is 1 except bit 3!
3. Bitwise AND with x:

$$\begin{array}{rr} x: & 00011101_2 \\ \& \quad \sim(1 \ll 3): & 11110111_2 \\ \hline \text{Result}: & 00010101_2 = 21_{10} \quad (29 - 8 = 21). \end{array}$$

3.2 Brian Kernighan's Trick: `x & (x - 1)`

This is the single most famous expression in all of bit manipulation.

Key Identity / Theorem: Clearing the Lowest Set Bit

For any non-zero integer x, the expression

$$x \,\&\, (x-1)$$

clears (turns to 0) the **lowest set bit** (rightmost 1) of x, leaving all higher bits completely unchanged.

Intuition & Mental Model: Why Subtracting 1 Targets Only the Rightmost 1

Suppose the lowest set bit of x is at index k. That means bit k is 1, and bits $0, 1, \ldots, k-1$ are all 0:

$$x = \underbrace{\text{prefix}}_{\text{unchanged}} \; \mathbf{1} \; \underbrace{00\ldots0}_{k \text{ zeros}}$$

When we subtract 1 from x, we need to borrow from the lowest available 1, which is at index k. By the geometric series identity $2^k - 1 = \sum_{i=0}^{k-1} 2^i$, borrowing from bit k turns bit k into 0 and turns all k trailing zeros into 1s:

$$x - 1 = \underbrace{\text{prefix}}_{\text{unchanged}} \; \mathbf{0} \; \underbrace{11\ldots1}_{k \text{ ones}}$$

Now take the bitwise AND of x and $x-1$:

- The prefix is identical in both, so prefix & prefix = prefix.
- Bit k is $1 \,\&\, 0 = 0$.
- The lowest k bits are $00\ldots0 \,\&\, 11\ldots1 = 00\ldots0$.

Thus, bit k vanishes and everything else stays untouched!

3.2.1 Direct Applications of `x & (x - 1)`

1. **Check if x is a Power of 2 in $O(1)$:** A positive integer is a power of 2 $(1, 2, 4, 8, \ldots)$ if and only if its binary representation has *exactly one set bit*. Clearing that single set bit must yield 0:

```cpp
bool isPowerOfTwo(long long x) {
    return x > 0 && (x & (x - 1)) == 0;
}
```

2. **Count Set Bits in O(number of set bits):** Instead of looping over all 64 bits, we can repeatedly strip the lowest set bit until x becomes 0:

```cpp
int countSetBits(long long x) {
    int count = 0;
    while (x > 0) {
        x &= (x - 1); // Clears lowest set bit in 1 operation
        count++;
    }
    return count;
}
```

3.3 Isolating the Lowest Set Bit: `x & -x` (The Fenwick Tree Trick)

What if instead of *clearing* the lowest set bit, we want to *extract* it (2^k) and zero out everything else?

Key Identity / Theorem: Isolating the Lowest Set Bit (`lowbit`)

For any integer x, the expression

$$\text{lowbit}(x) = x \,\&\, (-x)$$

returns a number with **only the lowest set bit of** x still set (i.e., 2^k, where $k = \text{ctz}(x)$).

Step-by-Step Dry Run: How `x & -x` Isolates the Rightmost 1 for $x = 40$ (`00101000`)

Let $x = 40 = 00101000_2$ (lowest set bit is $2^3 = 8$). Recall that in Two's Complement, $-x = \sim x + 1$:

1. Original x: $\texttt{00101000}_2$
2. Invert all bits ($\sim x$): $\texttt{11010111}_2$ (prefix flipped, bit k is 0, trailing zeros are 1s)
3. Add 1 ($-x = \sim x + 1$): $\texttt{11011000}_2$ (carries ripple through trailing 1s and stop at bit k!)
4. Take $x \,\&\, (-x)$:

$$\begin{array}{rr} x: & \texttt{00101000}_2 \\ \& \quad -x: & \texttt{11011000}_2 \\ \hline x \,\&\, (-x): & \texttt{00001000}_2 = 8_{10}. \end{array}$$

Notice why: the prefix of $-x$ is the exact bitwise inverse of x's prefix, so prefix& $\sim$ prefix $= 0$. The trailing zeros are 0 in both. Only bit k is 1 in both x and $-x$!

Where is this used? In a **Fenwick Tree (Binary Indexed Tree)**, every index i stores a range of length $\text{lowbit}(i) = i \,\&\, -i$. Moving to the parent node during an update is simply `i += (i & -i)`, and moving to the prefix predecessor during a query is `i -= (i & -i)` (which is identical to `i &= (i - 1)`!).

3.4 Complete Cheat Sheet of Single-Word Bit Hacks

Expression	**Effect on Binary Representation**	**Example ($x = \texttt{101100}_2$)**
`x & (x - 1)`	Clears the lowest set bit	$\texttt{101000}_2$
`x & -x`	Isolates the lowest set bit	$\texttt{000100}_2$
`x | (x + 1)`	Sets the lowest cleared bit ($0 \to 1$)	$\texttt{101101}_2$
`x & (x + 1)`	Clears all trailing 1s	($x = \texttt{101011}_2 \to \texttt{101000}_2$)
`x | (x - 1)`	Sets all trailing 0s to 1	$\texttt{101111}_2$
`~x & (x + 1)`	Isolates the lowest cleared bit	$\texttt{000001}_2$
`x & ((1LL << k) - 1)`	Extracts lowest k bits ($x \bmod 2^k$)	($k = 3 \to \texttt{000100}_2$)

Chapter 4

GCC Built-in Intrinsics, C++20 <bit>, & SWAR

Modern CPUs have dedicated single-cycle hardware instructions (`POPCNT`, `LZCNT`, `TZCNT`) for inspecting bits. In GCC/Clang, we access these via `__builtin_*` functions in $O(1)$ time.

4.1 The Essential GCC Intrinsics

32-bit (`unsigned int`)	64-bit (`unsigned long long`)	Description
`__builtin_popcount(x)`	`__builtin_popcountll(x)`	Number of set bits (1s) in x.
`__builtin_clz(x)`	`__builtin_clzll(x)`	Count of leading zeros (from MSB).
`__builtin_ctz(x)`	`__builtin_ctzll(x)`	Count of trailing zeros (from LSB).
`__builtin_ffs(x)`	`__builtin_ffsll(x)`	$1 + \text{ctz}(x)$ (or 0 if $x = 0$).
`__builtin_parity(x)`	`__builtin_parityll(x)`	popcount(x) mod 2 (1 if odd 1s).

Compiler & Overflow Trap: Intrinsics Trap: `clz(0)` and `ctz(0)` are Undefined Behavior!

Calling `__builtin_clz(0)`, `__builtin_clzll(0)`, `__builtin_ctz(0)`, or `__builtin_ctzll(0)` when $x = 0$ is **Undefined Behavior**! Always check `if (x == 0)` before calling them. Also, never pass a 64-bit `long long` to the 32-bit `__builtin_popcount(x)`—it will silently truncate the upper 32 bits! Always use the `ll` suffix for `long long`.

4.1.1 Computing $\lfloor \log_2(x) \rfloor$ and Highest Power of 2 in $O(1)$

Using floating-point `log2(x)` on 64-bit integers can fail due to 53-bit `double` mantissa precision loss. Instead, use `__builtin_clzll`:

- For a 32-bit positive `int` $x > 0$: $\lfloor \log_2(x) \rfloor = 31 - \texttt{__builtin_clz}(x)$.
- For a 64-bit positive `long long` $x > 0$: $\lfloor \log_2(x) \rfloor = 63 - \texttt{__builtin_clzll}(x)$.

4.1.2 Modern C++20 <bit> Equivalents

Since C++20, standard portable functions are available in `#include <bit>` (Note: they require **unsigned** integer types like `uint64_t` or `unsigned long long`):

```cpp
#include <bit>
#include <cstdint>

uint64_t x = 42;
int ones  = std::popcount(x);      // Number of set bits
int lzero = std::countl_zero(x);   // Safe on x = 0 (returns 64!)
int rzero = std::countr_zero(x);   // Safe on x = 0 (returns 64!)
int width = std::bit_width(x);     // 1 + floor(log2(x)), or 0 if x == 0
bool pow2 = std::has_single_bit(x); // True iff x is a power of 2
uint64_t f = std::bit_floor(x);    // Largest power of 2 <= x (32)
uint64_t c = std::bit_ceil(x);     // Smallest power of 2 >= x (64)
```

4.2 Under the Hood: SWAR (SIMD Within A Register)

How do you compute `popcount(x)` or reverse the bits of a 32-bit integer in $O(\log W) = 5$ steps without loops or hardware intrinsics? By using **SWAR**—treating a single 32-bit or 64-bit register as an array of small blocks and operating on all blocks in parallel using magic hexadecimal masks!

Intuition & Mental Model: Divide-and-Conquer Parallel Bit Counting

To count the 1s in a 32-bit integer x:

1. Pair up adjacent bits (b_{2i+1}, b_{2i}) and replace each 2-bit block with the sum of its two bits (0, 1, or 2).
2. Pair up adjacent 2-bit sums and add them into 4-bit sums (0 to 4).
3. Pair up adjacent 4-bit sums into 8-bit sums, then 16-bit, then 32-bit!

The magic masks select alternating blocks of sizes $1, 2, 4, 8, 16$:

- `0x55555555` = `01010101...`$_2$ (selects every even bit)
- `0x33333333` = `00110011...`$_2$ (selects alternating 2-bit blocks)
- `0x0F0F0F0F` = `00001111...`$_2$ (selects alternating 4-bit blocks)

```cpp
// Compute popcount of 32-bit unsigned int in 5 parallel steps (SWAR)
int swar_popcount(uint32_t x) {
    x = (x & 0x55555555U) + ((x >> 1) & 0x55555555U); // 2-bit sums
    x = (x & 0x33333333U) + ((x >> 2) & 0x33333333U); // 4-bit sums
    x = (x & 0x0F0F0F0FU) + ((x >> 4) & 0x0F0F0F0FU); // 8-bit sums
    x = (x & 0x00FF00FFU) + ((x >> 8) & 0x00FF00FFU); // 16-bit sums
    x = (x & 0x0000FFFFU) + ((x >> 16)& 0x0000FFFFU); // 32-bit total
        sum
    return x;
}

// Reverse all 32 bits of an unsigned integer in O(log W) = 5 steps!
uint32_t reverse_bits(uint32_t x) {
    x = ((x >> 1) & 0x55555555U) | ((x & 0x55555555U) << 1); // Swap
        adjacent 1-bit pairs
    x = ((x >> 2) & 0x33333333U) | ((x & 0x33333333U) << 2); // Swap
        adjacent 2-bit pairs
```

```
15      x = ((x >> 4) & 0x0F0F0F0FU) | ((x & 0x0F0F0F0FU) << 4); // Swap
            adjacent 4-bit nibbles
16      x = ((x >> 8) & 0x00FF00FFU) | ((x & 0x00FF00FFU) << 8); // Swap
            adjacent bytes
17      x = (x >> 16) | (x << 16);                               // Swap
            16-bit halves
18      return x;
19  }
```

Chapter 5

The Algebra of XOR & Bitwise Contribution Technique

XOR ($\oplus$) is the darling of Competitive Programming problem setters because unlike AND and OR (which lose information by irreversibly clamping bits to 0 or 1), **XOR is completely reversible**.

5.1 The Five Algebraic Laws of XOR

1. **Identity:** $x \oplus 0 = x$.
2. **Self-Inverse (Cancellation):** $x \oplus x = 0$. Every number cancels itself!
3. **Commutative & Associative:** $a \oplus b = b \oplus a$ and $(a \oplus b) \oplus c = a \oplus (b \oplus c)$.
    - *Consequence:* In a huge XOR sum $a_1 \oplus a_2 \oplus \cdots \oplus a_n$, order does not matter at all. Any value that appears an **even** number of times cancels out to 0; any value that appears an **odd** number of times survives once.
4. **Invertibility (Moving across the equals sign):**

$$a \oplus b = c \quad \iff \quad a = b \oplus c \quad \iff \quad b = a \oplus c.$$

5. **Prefix XOR Trick:** Because XOR has an inverse (itself!), we can compute the XOR of any subarray $A[L \dots R]$ in $O(1)$ time using a prefix XOR array $P[i] = A[1] \oplus \cdots \oplus A[i]$:

$$\bigoplus_{k=L}^{R} A[k] = P[R] \oplus P[L-1].$$

5.1.1 Classic Application: Finding Two Unique Numbers

Problem: In an array of N integers, every element appears an even number of times except for *two distinct numbers* u and v, which appear an odd number of times. Find u and v in $O(N)$ time and $O(1)$ extra space.

Solution:

1. XOR all elements of the array together. All paired elements cancel out, leaving $S = u \oplus v$.
2. Since $u \neq v$, we know $S \neq 0$, so S has at least one set bit. Isolate its lowest set bit using $\text{diff} = S \,\&\, (-S)$.

3. Because that bit is 1 in $S = u \oplus v$, one of $\{u, v\}$ must have that bit set to 1, and the other must have it set to 0!

4. Partition the array into two buckets based on whether `(x & diff) != 0`, and XOR each bucket separately. Duplicates always fall into the same bucket and cancel out, leaving u in one bucket and v in the other!

5.2 Fundamental Identities Connecting Addition and Bitwise Ops

How does regular arithmetic addition $(+)$ interact with $\&, |, \oplus$? These two identities appear in countless Codeforces Div. 1 / Div. 2 problems:

Key Identity / Theorem: The Carry-Addition & Symmetric Identities

For any two non-negative integers a and b:

$$a + b = (a \oplus b) + 2(a \,\&\, b) \tag{5.1}$$
$$a + b = (a \mid b) + (a \,\&\, b) \tag{5.2}$$

Intuition & Mental Model: Why $a + b = (a \oplus b) + 2(a \,\&\, b)$ Works

When you add two binary numbers column by column:

- $a \oplus b$ computes the sum at each bit position **ignoring carries** $(1+1 \equiv 0 \pmod 2)$.
- Where does a carry occur? A carry is generated at position i if and only if *both* a and b have a 1 at bit i—which is literally $a \,\&\, b$!
- Since a carry generated at bit i gets added to bit $i+1$, its value is shifted left by 1 position: $(a \,\&\, b) \ll 1 = 2(a \,\&\, b)$.

For Identity (5.2), look at each bit position i:

- If bits (a_i, b_i) are $(0, 0)$, both sides contribute 0.
- If bits (a_i, b_i) are $(1, 0)$ or $(0, 1)$, $a \mid b$ has 1 and $a \,\&\, b$ has 0, matching $1 + 0$.
- If bits (a_i, b_i) are $(1, 1)$, both $a \mid b$ and $a \,\&\, b$ have 1, matching $1 + 1 = 2$.

Important Corollary (Triangle Inequality for XOR): Since $2(a \,\&\, b) \geq 0$ for non-negative integers, Identity (5.1) immediately implies:

$$a \oplus b \leq a + b$$

with equality **if and only if** $a \,\&\, b = 0$ (meaning a and b share no common set bits!).

5.3 The Bit-by-Bit Contribution Technique

Whenever a problem asks you to compute the **sum** of a bitwise expression over all pairs, triplets, or subarrays (e.g., $\sum_{i<j}(A_i \oplus A_j)$ or $\sum_{i,j}(A_i \,\&\, A_j)$), trying to evaluate $A_i \oplus A_j$ as whole numbers takes $O(N^2)$ time.

The breakthrough is **Linearity of Expectation / Summation over Independent Bits**: Because any integer X can be decomposed into its individual bits as $X = \sum_{b=0}^{30} \text{bit}_b(X) \cdot 2^b$, we can swap the outer sum and the bit sum!

Key Identity / Theorem: Bit-by-Bit Independence Decomposition

$$\sum_{i=1}^{N}\sum_{j=i+1}^{N}(A_i \oplus A_j) = \sum_{b=0}^{30} 2^b \cdot (\text{Number of pairs } (i,j) \text{ where bit } b \text{ of } (A_i \oplus A_j) \text{ is } 1)$$

Step-by-Step Dry Run: Solving All-Pairs XOR Sum in $O(N \log(\max A))$

Suppose we look at a single bit position $b \in [0, 30]$ across all N elements of array A. Every element A_i either has bit b equal to 1 or 0. Let:

- c_1 = count of elements in A whose b-th bit is 1.
- $c_0 = N - c_1$ = count of elements in A whose b-th bit is 0.

How many pairs (i, j) with $i < j$ have bit b equal to 1 in:

1. $A_i \oplus A_j$? We need one 1 and one 0, so there are exactly $\mathbf{c_1 \cdot c_0}$ pairs!
2. $A_i \,\&\, A_j$? We need both bits to be 1, so there are $\binom{\mathbf{c_1}}{\mathbf{2}} = \frac{\mathbf{c_1(c_1-1)}}{\mathbf{2}}$ pairs!
3. $A_i \mid A_j$? All pairs except $(0, 0)$, so there are $\binom{\mathbf{N}}{\mathbf{2}} - \binom{\mathbf{c_0}}{\mathbf{2}}$ pairs!

Each such pair contributes 2^b to the total sum!

```cpp
long long sumOfAllPairsXOR(const vector<int>& A) {
    int n = A.size();
    long long total_sum = 0;
    for (int b = 0; b < 30; b++) {
        long long c1 = 0;
        for (int x : A) {
            if ((x >> b) & 1) c1++;
        }
        long long c0 = n - c1;
        long long pairs_with_bit_b = c1 * c0;
        total_sum += pairs_with_bit_b * (1LL << b);
    }
    return total_sum;
}
```

Chapter 6

Bitmasks for Sets, Submask Iteration, Gosper's Hack, & Gray Codes

When the universe of elements $U = \{0, 1, \ldots, n-1\}$ is small ($n \leq 60$), we can represent any subset $S \subseteq U$ as a single integer $mask \in [0, 2^n - 1]$, where bit i is 1 if and only if $i \in S$.

6.1 Set Theory via Bitmasks in $O(1)$

Set Notation	**Meaning**	**Bitmask Expression**
$\emptyset$	Empty set	`0`
$U = \{0, \ldots, n-1\}$	Universal set	`(1LL << n) - 1`
$\{i\}$	Singleton set	`1LL << i`
$i \in A$	Membership test	`((A >> i) & 1) != 0`
$\|A\|$	Cardinality (size)	`__builtin_popcountll(A)`
$A \cup B$	Union	`A | B`
$A \cap B$	Intersection	`A & B`
$A \setminus B$	Set Difference	`A & ~B`
$A \triangle B$	Symmetric Difference	`A ^ B`
$A \subseteq B$	Subset check	`(A & B) == A` (or `(A | B) == B`)

6.2 Iterating Over All Submasks of a Mask in $O(3^n)$

Suppose you are given a specific bitmask m (say $m = 10110_2$), and you want to iterate *only* over its valid submasks $s \subseteq m$ (masks that only have 1s where m also has 1s). If m has $k = \text{popcount}(m)$ set bits, there are 2^k valid submasks. Instead of looping from 0 to $2^n - 1$ and checking if `(s & m) == s`, we can jump **directly** from one valid submask to the next in $O(1)$ per submask!

Key Identity / Theorem: Descending Submask Enumeration Idiom

```
// Iterates over all non-empty submasks s of m in strictly
    decreasing order:
for (int s = m; s > 0; s = (s - 1) & m) {
    // Process submask s...
}
// Process s = 0 separately if needed.
```

Intuition & Mental Model: Why `s = (s - 1) & m` Jumps to the Exact Next Submask

We want the largest integer $s' < s$ such that $s' \subseteq m$.

1. Subtracting 1 $(s-1)$ clears the lowest set bit of s and sets all bits to its right to 1.
2. However, some of those newly set trailing 1s might be at positions where m has a 0 (invalid for a submask of m).
3. Taking bitwise AND with m (`(s - 1) & m`) immediately strips away those forbidden bits, leaving 1s only at valid positions of m to the right of the cleared bit!

Step-by-Step Dry Run: Submasks of $m = 10101_2$ (21_{10})

Watch how s steps through all $2^3 = 8$ submasks of $m = 10101_2$ without ever visiting an invalid mask:

Step	Current s	$s-1$	$s_{\text{next}} = (s-1)\ \&\ m$
1	10101_2 (21)	10100_2	10100_2 (20)
2	10100_2 (20)	10011_2	10001_2 (17)
3	10001_2 (17)	10000_2	10000_2 (16)
4	10000_2 (16)	01111_2	00101_2 (5)
5	00101_2 (5)	00100_2	00100_2 (4)
6	00100_2 (4)	00011_2	00001_2 (1)
7	00001_2 (1)	00000_2	00000_2 (0) $\rightarrow$ Loop terminates

6.2.1 Why Iterating Submasks for All Masks is $O(3^n)$, NOT $O(4^n)$!

Suppose we run an outer loop over all masks $m \in [0, 2^n - 1]$, and an inner loop over all submasks $s \subseteq m$:

```
for (int m = 0; m < (1 << n); m++) {
    for (int s = m; ; s = (s - 1) & m) {
        // O(1) work for pair (m, s) where s is a submask of m
        if (s == 0) break;
    }
}
```

A naive upper bound says: 2^n outer iterations $\times 2^n$ inner iterations $= O(4^n)$. **That bound is overly loose!** Let us find the exact number of iterations in two ways:

Proof 1 (Combinatorial / Bit-by-Bit View): For each bit position $i \in [0, n-1]$, what are the valid choices for the pair $(\text{bit}_i(m), \text{bit}_i(s))$ such that $s \subseteq m$?

- $\text{bit}_i(m) = 0, \text{bit}_i(s) = 0$ (Element i is in neither m nor s).
- $\text{bit}_i(m) = 1, \text{bit}_i(s) = 0$ (Element i is in m, but excluded from s).
- $\text{bit}_i(m) = 1, \text{bit}_i(s) = 1$ (Element i is in both m and s).

Notice $\text{bit}_i(m) = 0, \text{bit}_i(s) = 1$ is impossible! Since each of the n bit positions independently has **3 valid states**, the total number of pairs (m, s) is:

$$\underbrace{3 \times 3 \times \cdots \times 3}_{n \text{ times}} = 3^n.$$

Proof 2 (Algebraic via Binomial Theorem): How many masks $m \in [0, 2^n - 1]$ have exactly k set bits? There are $\binom{n}{k}$ such masks. And each mask with k set bits has exactly 2^k submasks. Summing over all k from 0 to n:

$$\text{Total Iterations} = \sum_{k=0}^{n} \binom{n}{k} 2^k \cdot 1^{n-k} = (2+1)^n = 3^n.$$

For $n = 15$, $4^{15} \approx 10^9$ (Time Limit Exceeded), whereas $3^{15} \approx 1.43 \times 10^7$ (runs in **0.03 seconds**!).

6.3 Gosper's Hack: Generating All k-Bit Masks in $O(\binom{n}{k})$

What if you want to iterate over all n-bit masks that have **exactly k set bits** in increasing numerical order?

Intuition & Mental Model: How Gosper's Hack Finds the Next k-Bit Mask

To find the smallest integer $x' > x$ with the exact same number of 1s:

1. Find the rightmost contiguous block of 1s in x.
2. Take the highest 1 in that block and shift it left by one position (into the lowest 0 just above that block).
3. Take all the remaining 1s from that block and pack them as far right (to the LSB) as possible to make the resulting number as small as possible!

Step-by-Step Dry Run: Gosper's Hack on $x = 01011100_2$

1. Isolate the lowest set bit: $c = x \,\&\, (-x) = 00000100_2$.
2. Add c to x to ripple-carry across the rightmost block of 1s:

   $$r = x + c = 01011100_2 + 00000100_2 = 01100000_2.$$

   Notice r already has the left-shifted 1 at bit 5!
3. Now how many 1s were in that block? Look at $r \oplus x = 00111100_2$. If we shift this right by 2 positions and divide by c, we push the remaining two 1s all the way to bit 0:

   $$\text{next_x} = (((r \oplus x) \gg 2)/c) \mid r = 00000011_2 \mid 01100000_2 = 01100011_2.$$

```cpp
// Iterate over all n-bit masks with exactly k set bits in O(nCr(n, k))
void gospersHack(int n, int k) {
    if (k == 0) return;
    int x = (1 << k) - 1; // Smallest mask with k ones: 00...011...1
    int limit = (1 << n);
    while (x < limit) {
        // Process mask x (guaranteed to have popcount(x) == k)

        int c = x & -x;
        int r = x + c;
        x = (((r ^ x) >> 2) / c) | r;
    }
}
```

6.4 Gray Codes (Reflected Binary Code)

A **Gray code** of length n is an ordering of all 2^n binary numbers from 0 to $2^n - 1$ such that **every two adjacent numbers differ in exactly one bit** (Hamming distance = 1).

Key Identity / Theorem: Closed-Form Gray Code Formula & Inverse

The i-th Gray code (0-indexed) is given in $O(1)$ by:

$$G(i) = i \oplus (i \gg 1)$$

Conversely, given a Gray code g, we can recover its index i by prefix-XORing its bits from MSB to LSB:

```cpp
int grayCode(int i) {
    return i ^ (i >> 1);
}

int inverseGrayCode(int g) {
    int i = 0;
    for (; g > 0; g >>= 1) {
        i ^= g;
    }
    return i;
}
```

Why does $G(i)$ differ from $G(i-1)$ in only one bit? Recall from Chapter 3 that incrementing $i - 1 \to i$ flips a contiguous suffix of k bits: $011\ldots1_2 \to 100\ldots0_2$. Therefore, $(i-1) \oplus i = 00\ldots011\ldots1_2$ (a block of k trailing 1s). Now apply the Gray code formula:

$$G(i-1) \oplus G(i) = ((i-1) \oplus i) \oplus (((i-1) \oplus i) \gg 1) = \texttt{0..0111..1}_2 \oplus \texttt{0..0011..1}_2 = \texttt{0..0100..0}_2$$

Every trailing 1 cancels with the shifted copy except the highest one! So $G(i-1)$ and $G(i)$ differ at **strictly one bit** (specifically, bit $\text{ctz}(i)$).

Chapter 7

Bitmask Dynamic Programming & SOS DP

7.1 Standard Bitmask DP Patterns

Whenever $N \leq 20$ and you need to find an optimal permutation, matching, or partition of N items, the standard factorial brute force $O(N!)$ is way too slow ($20! \approx 2.4 \times 10^{18}$). However, notice that when building a permutation of k elements, we rarely care about the exact order of all k previous elements—we only care about **which subset of elements has been used so far** (and maybe which element was placed last). That collapses $N!$ down to 2^N states!

7.1.1 Pattern 1: Traveling Salesperson Problem (TSP) in $O(N^2 2^N)$

Let $dp[\text{mask}][u]$ be the minimum cost to visit the subset of cities represented by mask, ending at city u (where bit u of mask is 1).

```cpp
const int INF = 1e9;
// dist[u][v] = edge weight between u and v
int tsp(int n, const vector<vector<int>>& dist) {
    vector<vector<int>> dp(1 << n, vector<int>(n, INF));
    dp[1 << 0][0] = 0; // Start at city 0, mask = {0}

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            // Try transitioning to an unvisited city v
            int unvisited = ((1 << n) - 1) ^ mask;
            for (int temp = unvisited; temp > 0; temp &= (temp - 1)) {
                int v = __builtin_ctz(temp);
                int next_mask = mask | (1 << v);
                dp[next_mask][v] = min(dp[next_mask][v], dp[mask][u] +
                    dist[u][v]);
            }
        }
    }
    int ans = INF, full_mask = (1 << n) - 1;
    for (int u = 1; u < n; u++) {
        ans = min(ans, dp[full_mask][u] + dist[u][0]);
    }
    return ans;
}
```

7.1.2 Pattern 2: Bipartite Assignment / Matching in $O(N2^N)$

Suppose we want to assign N workers to N tasks. A naive state is dp[worker_idx][task_mask], which has $N \cdot 2^N$ states. **Optimization Trick:** If we assign workers $0, 1, \ldots, N-1$ in order, the index of the current worker is automatically equal to the number of tasks already assigned:

$$\text{worker_idx} = \text{popcount}(\text{task_mask})!$$

This eliminates the first dimension completely, reducing memory and time to $O(N2^N)$!

7.2 Sum Over Subsets (SOS) DP: From $O(3^N)$ to $O(N2^N)$

The Problem: You are given an array A of size 2^N (indexed by N-bit masks). You want to compute the function F[mask] for all mask $\in [0, 2^N - 1]$, defined as the sum of $A[s]$ over all submasks $s \subseteq$ mask:

$$F[\text{mask}] = \sum_{s \subseteq \text{mask}} A[s] \quad \Longleftrightarrow \quad F[\text{mask}] = \sum_{(s \& \text{mask}) = s} A[s].$$

As we saw in Chapter 6:

- Naive double loop over all masks: $O(4^N)$ — passes for $N \leq 12$.
- Submask enumeration `for (s = m; s; s = (s-1)&m)`: $O(3^N)$ — passes for $N \leq 15$.
- What if $N = 20$? Then $3^{20} \approx 3.48 \times 10^9$ (TLE!), whereas $N2^N = 20 \times 2^{20} \approx 2 \times 10^7$ (passes effortlessly in 0.05s!). That is what **SOS DP** achieves.

Intuition & Mental Model: Why Submask Enumeration Wastes Work & How SOS DP Fixes It

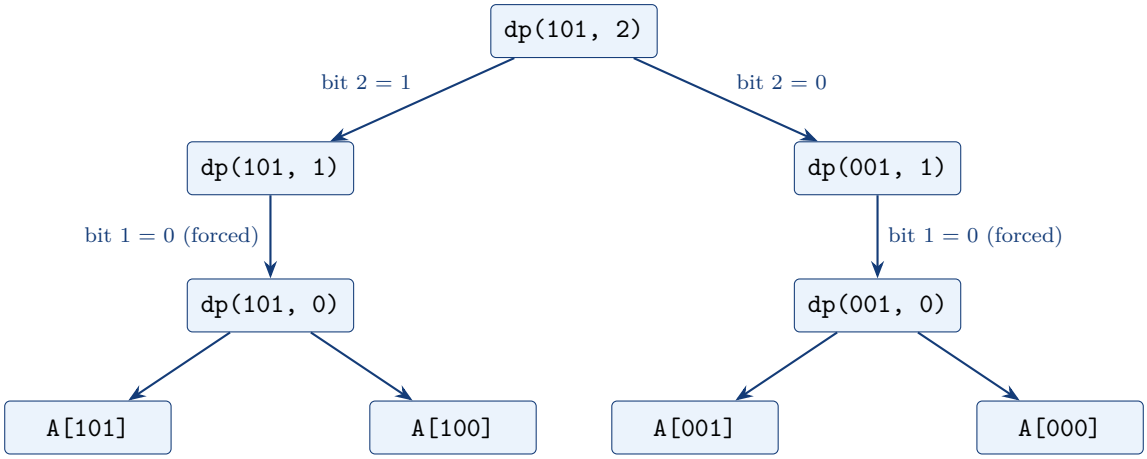
In $O(3^N)$ submask enumeration, every mask independently visits all its submasks from scratch. For example, both $m_1 = \texttt{1110}_2$ and $m_2 = \texttt{1111}_2$ separately sum over the 8 submasks of $\texttt{1110}_2$.
To share work across masks, SOS DP processes the subset condition **one bit position at a time**, from bit $i = 0$ to $i = N - 1$.
Define $dp(\text{mask}, i)$ as the sum of $A[s]$ over all submasks $s \subseteq$ mask that are **only allowed to differ from mask in the lowest $i + 1$ bits (bits $0, 1, \ldots, i$)**, while bits $i + 1 \ldots N - 1$ of s must match mask identically!

- **Case 1: The i-th bit of mask is 0.** Since $s \subseteq$ mask, the i-th bit of s *must also be* 0. There is no choice at bit i:

$$dp(\text{mask}, i) = dp(\text{mask}, i-1).$$

- **Case 2: The i-th bit of mask is 1.** Now a submask $s \subseteq$ mask can either have its i-th bit equal to 1 (matching mask) **OR** have its i-th bit equal to 0 (matching mask $\oplus\, 2^i$). These two groups are completely disjoint!

$$dp(\text{mask}, i) = \underbrace{dp(\text{mask}, i-1)}_{\text{bit } i \text{ of } s \text{ is } 1} + \underbrace{dp(\text{mask} \oplus (1 \ll i), i-1)}_{\text{bit } i \text{ of } s \text{ is } 0}.$$



7.2.1 The 7-Line In-Place 1D SOS DP Implementation

Because $dp(\text{mask}, i)$ only depends on stage $i-1$, and because $\text{mask} \oplus (1 \ll i)$ has its i-th bit equal to 0 (which was never modified during stage i!), we can drop the second dimension i and update $F[\text{mask}]$ **in-place** in ANY loop order for mask!

```
// Initialize F[mask] = A[mask] for all masks in [0, (1 << N) - 1]

// 1. Standard SOS DP: Sum over all SUBMASKS (s is a submask of mask)
for (int i = 0; i < N; i++) {
    for (int mask = 0; mask < (1 << N); mask++) {
        if (mask & (1 << i)) {
            F[mask] += F[mask ^ (1 << i)];
        }
    }
}

// 2. Dual SOS DP: Sum over all SUPERMASKS (mask is a submask of s)
for (int i = 0; i < N; i++) {
    for (int mask = (1 << N) - 1; mask >= 0; mask--) {
        if (!(mask & (1 << i))) {
            F[mask] += F[mask | (1 << i)];
        }
    }
}
```

Step-by-Step Dry Run: Tracing SOS DP for $N = 2$ on $A = \{A_0, A_1, A_2, A_3\}$

Initially, $F = [A_{00},\ A_{01},\ A_{10},\ A_{11}]$.

- **Outer loop $i = 0$ (processing bit 0):** Masks with bit 0 set are $01_2(1)$ and $11_2(3)$:

$$F[01] \leftarrow F[01] + F[00] = A_{01} + A_{00}$$
$$F[11] \leftarrow F[11] + F[10] = A_{11} + A_{10}$$

  Now $F = [A_{00},\ A_{00} + A_{01},\ A_{10},\ A_{10} + A_{11}]$.

- **Outer loop $i = 1$ (processing bit 1):** Masks with bit 1 set are $10_2(2)$ and $11_2(3)$:

$$F[10] \leftarrow F[10] + F[00] = A_{10} + A_{00}$$
$$F[11] \leftarrow F[11] + F[01] = (A_{10} + A_{11}) + (A_{00} + A_{01})$$

Every mask now holds the exact sum of its submasks!

7.2.2 OR / AND Convolutions via SOS DP (Fast Zeta & Möbius Transform)

Suppose you want to compute the **Bitwise OR Convolution** of two arrays A and B:

$$C[k] = \sum_{(i|j)=k} A[i] \cdot B[j].$$

Notice that $(i \mid j) \subseteq k \iff i \subseteq k$ and $j \subseteq k$! Therefore:

1. Apply Submask SOS DP (Fast Zeta Transform) to $A \to \hat{A}$ and $B \to \hat{B}$ in $O(N2^N)$.
2. Multiply point-wise: $\hat{C}[\text{mask}] = \hat{A}[\text{mask}] \cdot \hat{B}[\text{mask}]$. Now $\hat{C}[\text{mask}] = \sum_{s \subseteq \text{mask}} C[s]$.
3. Invert the SOS DP (Fast Möbius Transform) by simply replacing `+=` with `-=` in the SOS DP loop! For **Bitwise AND Convolution** ($\sum_{(i\&j)=k} A[i]B[j]$), do the exact same steps using **Supermask SOS DP**.

Chapter 8

Advanced Data Structures: `std::bitset`, Binary Tries, & XOR Basis

8.1 Bit-Parallel Speedups with `std::bitset` ($O(N/64)$)

C++'s `std::bitset<MAXN>` packs N boolean flags into an array of $\lceil N/64 \rceil$ 64-bit unsigned integers (`uint64_t`). Whenever you apply `&`, `|`, `^`, `<<`, or `>>` to a `std::bitset<N>`, the CPU processes **64 bits per instruction** (and with AVX2 auto-vectorization, 256 bits per instruction!).

This gives a massive **64× constant-factor speedup**, turning an $O(N^2)$ algorithm for $N = 50,000$ ($\approx 2.5 \times 10^9$ ops) into $\frac{N^2}{64} \approx 3.9 \times 10^7$ ops, which easily passes in 0.1 seconds!

8.1.1 Undocumented GCC Power Functions on `std::bitset`

Besides standard `.count()` (which calls hardware `popcount` on each 64-bit word), GCC's `std::bitset` provides two extraordinarily useful methods to iterate only over the 1-bits in $O(\text{number of 1s} + N/64)$ time:

- `bs._Find_first()`: Returns the index of the first set bit (or `bs.size()` if all bits are 0).
- `bs._Find_next(i)`: Returns the index of the first set bit strictly after index `i` (or `bs.size()` if none).

8.1.2 Classic Bitset Applications

1. **Subset Sum / 0-1 Knapsack Feasibility in $O(N \cdot W/64)$:** Normally, checking which subset sums up to W can be formed by N items takes $O(N \cdot W)$ boolean DP transitions: `dp[s] |= dp[s - w]`. With `std::bitset`, shifting the entire DP state by w and ORing takes a single line!

```
bitset<1000001> dp;
dp[0] = 1; // Sum 0 is always reachable
for (int w : weights) {
    dp |= (dp << w); // Updates all 1,000,000 sums in 1000000/64 =
       15,625 ops!
}
```

2. **Counting Triangles in an Undirected Graph in $O(V^3/64)$:** Store each vertex u's adjacency list as a `bitset<MAXV> adj[u]`. For every edge (u, v), the number of common neighbors w forming a triangle (u, v, w) is simply `(adj[u] & adj[v]).count()`! Dividing the total sum over all edges by 3 yields the triangle count.

8.2 The Binary Trie (0-1 Trie)

When a problem asks questions about **pairwise XOR** or **XOR inequalities** such as:

- *"Given a dynamic set S and a query integer x, find* $\max_{y \in S}(y \oplus x)$*"*, or
- *"Count how many $y \in S$ satisfy $(y \oplus x) < K$"*,

the **Binary Trie** is your go-to data structure.

Intuition & Mental Model: Strings of 30 Bits from MSB to LSB

Treat every 30-bit integer as a fixed-length string of characters '0' and '1', starting from the **Most Significant Bit (bit 29) at the root** down to the **Least Significant Bit (bit 0) at the leaves**.
Why must the root start at the MSB? Because of our geometric series theorem ($2^k > \sum_{i=0}^{k-1} 2^i$)! To maximize $y \oplus x$, we inspect x from bit 29 down to 0. At layer b, if bit b of x is *bit*, we greedily want to step into child $1 - bit$ so that their XOR has a 1 at bit b. Only if child $1 - bit$ does not exist (or has count 0) are we forced to take child *bit*.

8.2.1 Complete Production C++ Binary Trie Template

```
struct BinaryTrie {
    static const int LOG = 30; // Use 60 for 64-bit integers
    struct Node {
        int child[2] = {-1, -1};
        int cnt = 0; // Number of active integers passing through this
            node
    };
    vector<Node> tree;

    BinaryTrie() { tree.emplace_back(); } // Root is index 0

    // Insert (delta = +1) or Erase (delta = -1) integer x in O(LOG)
    void update(int x, int delta) {
        int u = 0;
        tree[u].cnt += delta;
        for (int b = LOG - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (tree[u].child[bit] == -1) {
                tree[u].child[bit] = tree.size();
                tree.emplace_back();
            }
            u = tree[u].child[bit];
            tree[u].cnt += delta;
        }
    }

    // Find max_{y in S} (y ^ x) in O(LOG)
    int queryMaxXor(int x) const {
        int u = 0, ans = 0;
        for (int b = LOG - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            int desired = 1 - bit; // Opposite bit gives 1 in XOR
            int nxt = tree[u].child[desired];
            if (nxt != -1 && tree[nxt].cnt > 0) {
                ans |= (1 << b);
```

```
                u = nxt;
            } else {
                u = tree[u].child[bit];
            }
        }
        return ans;
    }

    // Count how many y in S satisfy (y ^ x) < K in O(LOG)
    int countXorLessThan(int x, int K) const {
        int u = 0, ans = 0;
        for (int b = LOG - 1; b >= 0 && u != -1; b--) {
            int bit_x = (x >> b) & 1;
            int bit_k = (K >> b) & 1;
            if (bit_k == 1) {
                // If we pick y's bit == bit_x, then (y ^ x) has 0 at
                    bit b < 1 (bit_k),
                // so ALL elements in child[bit_x] are strictly less
                    than K!
                int same = tree[u].child[bit_x];
                if (same != -1) ans += tree[same].cnt;
                // Then continue down the branch where (y ^ x) matches
                    K at bit b (1)
                u = tree[u].child[1 - bit_x];
            } else {
                // bit_k == 0: (y ^ x) MUST have 0 at bit b to not
                    exceed K
                u = tree[u].child[bit_x];
            }
        }
        return ans;
    }
};
```

8.3 Linear XOR Basis (Gaussian Elimination over $\mathbb{F}_2^d$)

Wait: the Binary Trie solves queries over **single elements** $y \in A$ (or subarrays via prefix XORs). What if a problem asks about **XOR of arbitrary SUBSETS** of an array $A = [a_1, a_2, \ldots, a_n]$? For $n = 10^5$, there are $2^{100,000}$ subsets! How can we possibly answer questions about all 2^n subset XORs?

Intuition & Mental Model: Integers as Vectors in $\mathbb{F}_2^d$

Every d-bit non-negative integer x can be viewed as a d-dimensional vector over the finite field $\mathbb{F}_2 = \{0, 1\}$, where vector addition is bitwise XOR ($\oplus$) and scalar multiplication is bitwise AND with 0 or 1.
For example, $13 = 1101_2$ is the vector $(1, 1, 0, 1) \in \mathbb{F}_2^4$. Taking the XOR of a subset of numbers in A is literally taking a **linear combination** of vectors in $\mathbb{F}_2^d$!
By basic linear algebra, any subspace of $\mathbb{F}_2^d$ has dimension at most d ($d \leq 60$ for `long long`!). That means out of $n = 100,000$ numbers, we can extract a **Basis B of at most $d \leq 60$ numbers** that spans the *exact same set of XOR values* as the entire 100,000-element array!

8.3.1 Upper-Triangular Echelon Form & Online Insertion

We maintain an array `basis[0..d-1]` initialized to 0. **Invariant:** If `basis`$[i] \neq 0$, then the **Highest Set Bit (MSB)** of `basis`$[i]$ is at bit position i. Because every non-zero basis vector has a *strictly different* MSB, they are guaranteed to be linearly independent!

How do we insert a new number x into our basis? We iterate from the highest bit $i = d - 1$ down to 0:

1. If bit i of x is 0, continue to the next bit.
2. If bit i of x is 1:
   - **Case A (`basis`$[i] == 0$):** No basis vector currently controls bit i! So x cannot be formed by lower basis vectors. Set `basis`$[i] = x$, increment our basis size $|B|$, and return!
   - **Case B (`basis`$[i] \neq 0$):** Eliminate bit i from x by setting $x \leftarrow x \oplus$ `basis`$[i]$, and keep going down!
3. If x becomes 0, that means the original x was already a linear combination (XOR sum) of existing basis vectors, so it is linearly dependent and adds no new XOR capabilities.

Step-by-Step Dry Run: Building an XOR Basis from $A = \{12, 10, 6, 5\}$ ($d = 4$ bits)

In binary: $12 = \texttt{1100}_2,\ 10 = \texttt{1010}_2,\ 6 = \texttt{0110}_2,\ 5 = \texttt{0101}_2$.

1. **Insert** 12 **(`1100`):** Highest set bit is $i = 3$. Since `basis`$[3] == 0$, set `basis`$[3] = \texttt{1100}_2(12)$.
2. **Insert** 10 **(`1010`):**
   - At $i = 3$: bit 3 is 1, and `basis`$[3] = \texttt{1100}_2$ exists! Eliminate: $x \leftarrow \texttt{1010}_2 \oplus \texttt{1100}_2 = \texttt{0110}_2(6)$.
   - At $i = 2$: bit 2 of $x = \texttt{0110}_2$ is 1, and `basis`$[2] == 0$. Set `basis`$[2] = \texttt{0110}_2(6)$!
3. **Insert** 6 **(`0110`):**
   - At $i = 2$: bit 2 is 1, and `basis`$[2] = \texttt{0110}_2$ exists! Eliminate: $x \leftarrow \texttt{0110}_2 \oplus \texttt{0110}_2 = 0$.
   - x became 0! Indeed, $6 = 12 \oplus 10$, so 6 is redundant!
4. **Insert** 5 **(`0101`):**
   - At $i = 2$: bit 2 is 1 $\implies x \leftarrow \texttt{0101}_2 \oplus \texttt{0110}_2 = \texttt{0011}_2(3)$.
   - At $i = 1$: bit 1 of $x = \texttt{0011}_2$ is 1, and `basis`$[1] == 0$. Set `basis`$[1] = \texttt{0011}_2(3)$!

Final Basis: `basis`$[3] = \texttt{1100}_2$, `basis`$[2] = \texttt{0110}_2$, `basis`$[1] = \texttt{0011}_2$ (size $|B| = 3$).

8.3.2 The Five Fundamental Theorems of XOR Basis

Suppose an array A of n elements produces an XOR basis B of size $|B| = sz$.

1. **Number of Distinct Subset XORs:** Exactly $\mathbf{2^{sz}}$ distinct values can be formed.
2. **Uniform Multiplicity Theorem (Crucial!):** Every representable XOR value in $\text{span}(B)$ is formed by **the exact same number of subsets of A**, namely $\mathbf{2^{n-sz}}$ subsets!

- *Why?* Pick the $n - sz$ elements outside the basis arbitrarily (2^{n-sz} ways). Whatever their XOR sum is, there is a **unique** combination of the sz independent basis elements that adjusts that XOR sum to equal your target X!

3. **Reachability Check:** A target X can be formed as a subset XOR of A if and only if reducing X against the basis results in 0.

4. **Maximum Subset XOR:** Start with ans $= 0$. For $i = d - 1$ down to 0, greedily set ans $= \max(\text{ans}, \text{ans} \oplus \texttt{basis}[i])$. Since $\texttt{basis}[i]$ is the only remaining vector that can flip bit i, this greedy choice is 100% optimal!

5. **k-th Smallest Distinct Subset XOR:** First convert the basis into **Reduced Row Echelon Form (RREF)** so that if $\texttt{basis}[i] \neq 0$, no other basis vector has bit i set. Then the j-th non-zero basis vector directly corresponds to the j-th bit of $(k - 1)$!

8.3.3 Complete Grandmaster C++ `XorBasis` Struct

```cpp
struct XorBasis {
    static const int D = 60; // Use 30 for 32-bit ints, 60 for long long
    long long basis[D] = {0};
    int sz = 0; // Number of linearly independent vectors in the basis

    // Insert x into the basis in O(D). Returns true if x was independent.
    bool insert(long long x) {
        for (int i = D - 1; i >= 0; i--) {
            if (!((x >> i) & 1)) continue;
            if (!basis[i]) {
                basis[i] = x;
                sz++;
                return true;
            }
            x ^= basis[i];
        }
        return false; // x is already in the span of existing basis vectors
    }

    // Check if x can be formed as the XOR of some subset
    bool canForm(long long x) const {
        for (int i = D - 1; i >= 0; i--) {
            if ((x >> i) & 1) x ^= basis[i];
        }
        return x == 0;
    }

    // Return the maximum possible XOR of any subset (optionally XORed with initial val)
    long long getMaxXor(long long initial = 0) const {
        long long res = initial;
        for (int i = D - 1; i >= 0; i--) {
            res = max(res, res ^ basis[i]);
        }
        return res;
    }

```

```
    // Return the k-th smallest distinct subset XOR (1-indexed, where
       k = 1 gives 0)
    long long kthSmallestXor(long long k) const {
        if (k < 1 || k > (1LL << sz)) return -1; // Out of range
        k--; // Convert to 0-indexed binary representation

        // Convert to Reduced Row Echelon Form (diagonalize)
        long long rref[D];
        for (int i = 0; i < D; i++) rref[i] = basis[i];
        for (int i = 0; i < D; i++) {
            if (!rref[i]) continue;
            for (int j = i + 1; j < D; j++) {
                if ((rref[j] >> i) & 1) rref[j] ^= rref[i];
            }
        }

        long long ans = 0;
        for (int i = 0; i < D; i++) {
            if (!rref[i]) continue;
            if (k & 1) ans ^= rref[i];
            k >>= 1;
        }
        return ans;
    }

    // Merge another XorBasis into this one in O(D^2) (Useful for
       Segment Trees!)
    void merge(const XorBasis& other) {
        for (int i = 0; i < D; i++) {
            if (other.basis[i]) insert(other.basis[i]);
        }
    }
};
```

Chapter 9

Problem-Solving Decision Matrix & Practice Roadmap

When you encounter a bitwise problem during a contest, use this diagnostic table to immediately zero in on the right technique:

Clue in Problem Statement	Primary Technique to Apply
Sum of $(A_i \oplus A_j)$, $(A_i \,\&\, A_j)$, or subarray XOR/AND/OR sums	**Bit-by-Bit Contribution:** Solve each bit $b \in [0, 30]$ independently and multiply by 2^b.
Equation mixing $+$ and $\oplus$ or & (e.g., $a + b = X$ and $a \oplus b = Y$)	Use $a + b = (a \oplus b) + 2(a \,\&\, b) \implies a \,\&\, b = \frac{X-Y}{2}$, then check if $(a \oplus b) \,\&\, (a \,\&\, b) == 0$.
$N \leq 20$, find optimal order, matching, or subset partitioning	**Bitmask DP** ($O(N2^N)$ or $O(N^2 2^N)$) or **Submask Iteration** ($O(3^N)$ if $N \leq 15$).
$A_i \leq 10^6$ ($\approx 2^{20}$), find pair (A_i, A_j) with $A_i \,\&\, A_j = 0$ or $A_i \mid A_j = X$	**SOS DP:** $A_i \,\&\, A_j = 0 \iff A_j \subseteq \sim A_i$ (complement mask!). Compute submask DP in $O(B2^B)$ where $B = 20$.
Max/Min XOR of two elements $A_i \oplus A_j$, or count pairs with $A_i \oplus A_j < K$	**Binary Trie (0-1 Trie):** Walk MSB to LSB in $O(\log(\max A))$ per element.
Max/Min XOR of an *arbitrary subset*, or count subsets with XOR $= X$	**Linear XOR Basis ($\mathbb{F}_2^d$):** Reduce N elements to ≤ 60 basis elements in $O(N \log(\max A))$.
Range queries on subarrays for Subset XOR ($[L, R]$ queries)	**Segment Tree of XOR Bases** ($O(Q \log N \cdot D^2)$) or **Prefix Rightmost-Index XOR Basis** ($O((N + Q) \log(\max A))$).
Graph reachability, triangle counting, or 0-1 Knapsack with $N, W \leq 50,000$	**Bit-Parallel `std::bitset<N>`:** Pack boolean transitions into 64-bit operations for a 64× speedup.

9.1 Curated Practice Problem Set (Sorted by Rating)

1. **[800 – 1100] Foundations & Bit Tricks:**
   - *CSES Introductory:* Bit Strings, Gray Code
   - *Codeforces 579A:* Raising Bacteria (`__builtin_popcount`)
   - *Codeforces 1097B:* Petr and a Combination Lock (Brute force 2^N masks)

- *Codeforces 1514B:* AND 0, Sum Big (Bitwise independence & combinatorics)

2. [1200 – 1500] XOR Properties & Bit-by-Bit Contribution:

- *Codeforces 1325D:* Ehab the Xorcist ($a + b = (a \oplus b) + 2(a \,\&\, b)$ construction)
- *Codeforces 1879D:* Sum of XOR Functions (Prefix XOR + Bit-by-Bit contribution)
- *Codeforces 276D:* Little Girl and Maximum XOR (MSB divergence greedy)

3. [1600 – 1900] Bitmask DP, Binary Trie, & Submasks:

- *CSES Dynamic Programming:* Hamiltonian Flights, Elevator Rides, Counting Tilings
- *Codeforces 706D:* Vasiliy's Multiset (Classic dynamic Binary Trie)
- *Codeforces 817E:* Choosing The Commander (Binary Trie with $\oplus < K$ subtree counts)

4. [2000 – 2400+] SOS DP & Linear XOR Basis:

- *Codeforces 165E:* Compatible Numbers (Classic SOS DP introduction: $A_i \,\&\, A_j = 0$)
- *Codeforces 383E:* Vowels (SOS DP with Inclusion-Exclusion / Complement Principle)
- *Codeforces 449D:* Jzzhu and Numbers (Supermask SOS DP + Fast Möbius inversion)
- *CSES Additional Problems:* Maximum Xor Subset (Pure XOR Basis)
- *Codeforces 959F:* Mahmoud and Ehab and yet another xor task (XOR Basis + 2^{n-sz} multiplicity theorem)
- *Codeforces 1100F:* Ivan and Burgers (Range Max Subset XOR via Prefix Rightmost-Index XOR Basis)